

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ



ПРОГРАМ ХАНГАМЖИЙН ТӨСЛИЙН БАРИМТ БИЧИГ

SEMINAR 5 — OPENAPI 3.0 БА CODE SAMPLE

Хичээл:	ICSI438 — Software Project Documentation
Үндсэн төсөл:	Nogoolin
Хичээлийн багш:	Ph.D. Batnyam Battulga
Гүйцэтгэсэн:	Програм хангамжийн IV ангийн оюутан Ц. Тэнгис (22B1NUM6249)
Баримтын хувилбар:	v0.1 draft

Улаанбаатар

2026 он

Seminar 5 — OpenAPI 3.0 ба code sample

Төслийн GitHub дахь Sem5 хавтас: <https://github.com/Tengis01/Nogoolin/tree/main/docs/ICSI438/sem5>. Түүний renderers/ хавтаст Redoc-ийн redoc.html болон Swagger UI-ийн swagger.html байна.

Зорилго ба хүрээ

Nogoolin-ийн одоо кодоод бүртгэлтэй таван үйлдлийг OpenAPI 3.0.3 YAML-д тусгав. Сонголт нь Sem3-ын inquiry SRS, Sem4-ийн inquiry runtime view-тэй шууд холбогдоно. Энэ нь docs/phase-0/06-api-сpec.yaml-ийн бүх API-г орлуулах шинэ үндсэн spec биш; W5 лабораторийн нарийвчилсан slice юм. Серверийн URL нь localhost тул Swagger-ийн “Try it out” зөвхөн local API, local Supabase, тохирох auth token бэлэн үед ажиллана.

Үйлдэл	Sem3 шаардлага	Sem4 холбоо	Оролт	Амжилт
GET /products	Каталогийн контекст	C4-02 API	query	200 ProductList
POST /inquiries	FR-01/02/03/05	C4-D01	JSON body; JWT сонголтот	201 InquiryOne
GET /inquiries/mine	FR-06, NFR-01	C4-D01	JWT	200 InquiryArray
GET /admin/inquiries	FR-07/08	C4-D03	query + admin JWT	200 InquiryList
PATCH status	FR-09	C4-D03	path + JSON body + admin JWT	200 InquiryOne

PATCH status нь PATCH /admin/inquiries/{id}/status гэсэн бүтэн замтай. GET үйлдэлд requestBody бичээгүй. Оролтгүй GET /inquiries/mine нь JWT-ээр хэрэглэгчээ танина; бусад GET-ийн filter-ийг query parameter-аар өгнө. POST/PATCH-ийн жишээ болон response schema YAML-д байна. 400/401/403/404/429/500 нь тухайн үйлдэлд хэрэгтэй үед schema-тай. YAML дахь жишээ ID, огноо нь **схемийг тайлбарлах өгөгдөл**; серверээс авсан JSON биш. product холбоос response-д null эсвэл объект байх боломжтой; кодын join-оор нягтална.

Нэг drift илэрсэн: Sem3 FR-03 inquiry утсыг 8 цифр гэж шаарддаг, харин одоогийн phoneSchema нь +, зай, - тэмдэг оролцсон 8–15 тэмдэгт зөвшөөрдөг. W5 YAML нь **одоогийн API кодын** дүрмийг тусгасан; SRS ба кодын аль нэгийг багш/төслийн шийдвэрээр дараа нь нэг мөр болгох хэрэгтэй.

UE-5 — Corg.ly Python samples

Bhatti Ch.5 х.86–94-ийн контекст, товч байдал, нэршил, шууд ашиглалт, бодит хариунд нийцэх гэсэн таван хэмжүүрээр гурван тусдаа Python script бэлдэв. Эдгээр нь handout-ийн сургалтын **Corg.ly scenario**; Corg.ly-ийн ажилладаг сервер, батлагдсан request field contract, token бидэнд байхгүй. Тиймээс Python синтаксийг шалгасан ч endpoint-уудыг live ажиллуулсан, статус эсвэл returned JSON баталсан гэж үзэхгүй. Script нь CORGLY_BASE_URL, CORGLY_TOKEN environment variable шаардаж, зөвхөн хэрэглэгч гараар ажиллуулбал HTTP хүсэлт явуулна.

Зураг илгээх

upload_photo.py нь бүртгэлтэй pet ID болон локал JPEG файлыг multipart-аар илгээнэ. CLI-д photo ба --pet-id дамжуулна; photo field нэр нь handout-д тодорхойгүй тул серверийн жинхэнэ contract-той тулгах шаардлагатай.

```
"""UE-5 Corg.ly exercise: upload one pet image. Contract is unverified."""
```

```
import argparse
import os
from pathlib import Path

import requests
```

```

def main() -> None:
    parser = argparse.ArgumentParser(description="Upload a pet photo to a configured lab server")
    parser.add_argument("photo", type=Path, help="Existing pet image path")
    parser.add_argument("--pet-id", required=True, help="Registered pet ID")
    args = parser.parse_args()
    base_url = os.environ["CORGLY_BASE_URL"].rstrip("/")
    token = os.environ["CORGLY_TOKEN"]
    with args.photo.open("rb") as image:
        response = requests.post(
            f"{base_url}/v1/pets/upload-photo",
            headers={"Authorization": f"Bearer {token}"},
            data={"pet_id": args.pet_id},
            files={"photo": (args.photo.name, image, "image/jpeg")},
            timeout=20,
        )
    print(response.status_code, response.text)
    response.raise_for_status()

if __name__ == "__main__":
    main()

```

Bark орчуулах

translate_bark.py нь pet ID болон локал WAV бичлэгийг multipart-аар илгээнэ. audio field нэр болон MIME нь лабораторийн боломжит таамаг тул Corg.ly API-ийн баталгаа биш.

""UE-5 Corg.ly exercise: request bark translation. Contract is unverified.""

```

import argparse
import os
from pathlib import Path

import requests

def main() -> None:
    parser = argparse.ArgumentParser(description="Send a bark recording to a configured lab server")
    parser.add_argument("audio", type=Path, help="Existing WAV recording path")
    parser.add_argument("--pet-id", required=True, help="Registered pet ID")
    args = parser.parse_args()
    base_url = os.environ["CORGLY_BASE_URL"].rstrip("/")
    token = os.environ["CORGLY_TOKEN"]
    with args.audio.open("rb") as recording:
        response = requests.post(
            f"{base_url}/v1/audio/translate-bark",
            headers={"Authorization": f"Bearer {token}"},
            data={"pet_id": args.pet_id},
            files={"audio": (args.audio.name, recording, "audio/wav")},
            timeout=30,
        )
    print(response.status_code, response.text)
    response.raise_for_status()

if __name__ == "__main__":
    main()

```

Webhook бүртгүүлэх

subscribe_webhook.py нь pet ID ба өөрийн эзэмшлийн HTTPS callback URL-ийг JSON-аар илгээнэ. Callback endpoint-ийг олон нийтэд ажиллуулж туршаагүй; url field-ийн нэрийг мөн бодит contract-той шалгах хэрэгтэй.

"""UE-5 Corg.ly exercise: subscribe to pet events. Contract is unverified."""

```
import argparse
import os
from urllib.parse import urlparse

import requests

def main() -> None:
    parser = argparse.ArgumentParser(description="Register an HTTPS webhook with a lab server")
    parser.add_argument("webhook_url", help="HTTPS callback owned by the caller")
    parser.add_argument("--pet-id", required=True, help="Registered pet ID")
    args = parser.parse_args()
    if urlparse(args.webhook_url).scheme != "https":
        parser.error("webhook_url must use HTTPS")
    base_url = os.environ["CORGLY_BASE_URL"].rstrip("/")
    token = os.environ["CORGLY_TOKEN"]
    response = requests.post(
        f"{base_url}/v1/webhooks/subscribe",
        headers={"Authorization": f"Bearer {token}"},
        json={"pet_id": args.pet_id, "url": args.webhook_url},
        timeout=20,
    )
    print(response.status_code, response.text)
    response.raise_for_status()

if __name__ == "__main__":
    main()
```

Code sample audit

Оноо 1–5; хамгийн бага нотолгоотой Trustworthy хэмжүүрийг өндөр оноогоор нөхөөгүй. Bhatti х.89–94-ийн request/response-ийн яг тохирол, товч, ойлгомжтой, солих утга тодорхой байх шалгуурыг хэрэглэв.

Sample	Explained	Concise	Clear	Usable	Trustworthy	Дундаж
Photo upload	5	4	5	3	1	3.6
Bark translation	5	4	5	3	1	3.6
Webhook subscribe	5	4	5	3	1	3.6
Nogoolin OpenAPI request/response examples	4	5	4	3	2	3.6
Нийт						3.6/5

Өндөр үнэлгээний үндэслэл: зорилго, оролт, тайлбартай нэр, богино API call бий; foo/bar/test орлуулагч байхгүй. Бага үнэлгээний үндэслэл: Corg.ly request field болон response shape бодитоор батлагдаагүй, Nogoolin example JSON нь код/схемээр нягталсан загвар боловч live capture биш. Энэ нь DoD-ийн 4/5 босгыг **одоогоор хангахгүй**. Бодит sandbox ирвэл field contract-ийг тулгаж, response-ийг capture хийж, дахин үнэлнэ.

Swagger UI ба Redoc — шийдвэрийн тайлан (100 үг)

Swagger UI нь endpoint бүрийн параметр, хамгаалалт, body-г бөглөж хүсэлт илгээх боломжтой тул хөгжүүлэгчийн локал туршилтад тохирно. Гэхдээ “Try it out” нь токен, ажиллаж буй API, өгөгдлийн

сан шаардана; нийтэд нээлттэй орчинд санамсаргүй бичилт үүсгэж болно. Redoc нь уншихад цэгцтэй navigation, schema холбоос, гурван баганат үзэмжтэй тул хэрэглэгчид API гэрээг тайлбарлахад илүү тохиромжтой. Хайлт ба байршлын хурдыг endpoint дээр хэмжээгүй. Одоохондоо Redoc-ийг унших үндсэн хувилбар, Swagger UI-г зөвхөн локал sandbox гэж сонгов. Хоёр renderer-ийг ижил YAML-аас үүсгэвэл зөрүү багасна. Public URL, хэрэглэгчийн үнэлгээ байхгүй тул энэ шийдвэрийг түр гэж үзэж, deployment-ийн дараа дахин шалгана. Энэ нь лабораторийн хүрээний санал юм.

Reflection

Bhatti Ch.5 x.89–94-ийн хамгийн чухал санаа нь response жишээ код шигээ шалгагдах ёстой гэдэг байлаа. Өмнөх spec-д жишээ өгөгдөл байхад түүнийг бодит үр дүн гэж андуурах амархан. Би энэ удаа жишээг зориуд “illustrative” гэж тэмдэглэж, өөрийн хийж чадаагүй live шалгалтыг ил үлдээв. Chinchilla Ch.6 x.79-ийн docs-as-code ба render-ийн санаагаар YAML-г локал lint, standalone HTML, PDF тайлантай нэг эхээс холбоно. Миний төслийн хамгийн том зөрүү нь хуучин Phase-0 spec-ийн зарим response жишээ бодит controller-ийн return envelope-той бүрэн нийцэхгүй байгаа явдал; W5-ийн таван route-ыг кодоос дахин тулгасан ч бүх API-ийн drift audit хийгээгүй.

Нотолгоо ба дутуу шалгуур

- redocly lint: OpenAPI valid, алдаа 0; localhost болон лиценз metadata-ийн warning-ийг README-д тэмдэглэв.
- python3 -m py_compile: гурван script синтаксийн хувьд зөв.
- Redoc standalone HTML болон Swagger UI local HTML-г бэлдэв. Static asset замыг шалгасан ч browser rendering, “Try it out” runtime туршилт хийгдээгүй.
- Public хоёр URL, workspace upload, Git commit/push, CI executable bonus, Corg.ly live response: хийгдээгүй.

Эх: W5 тусгай handout; Bhatti et al., *Docs for Developers*, Ch.5 x.86–94; Chinchilla, *Technical Writing for Software Developers*, Ch.6 x.79.